



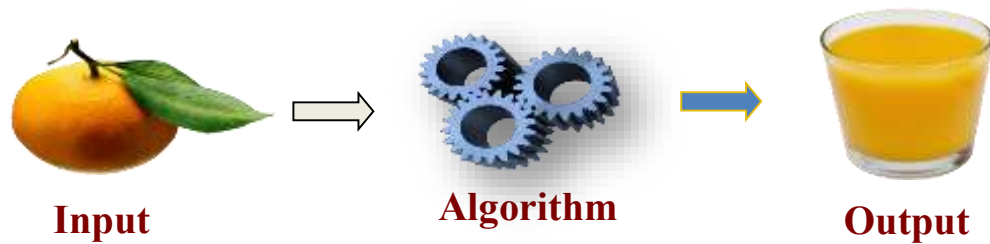
Data Structure Using C++

Analysis of Algorithms

Dr. Ali Hamzah



Analysis of Algorithms

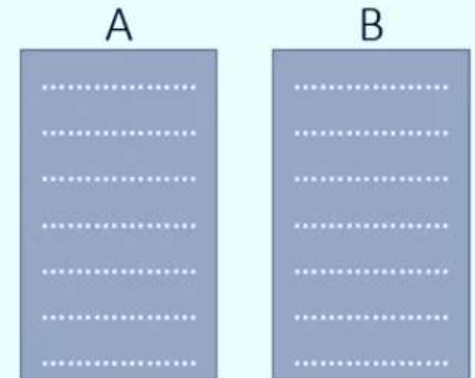


Our goal:

Analysis of Algorithms
Measure Performance

Is this algorithm good ?

- Correct
- Efficient
 - Time Complexity
 - Space Complexity
- Easy to implement



Limitations of Experiments

- It is necessary to implement the whole **algorithm** before conducting any experiment, which may be difficult.
- Results may not be indicative of the running time on **other inputs not included** in the experiment.
- In order to compare two algorithms, the **same hardware and software environments** must be used



Time complexity analysis

- Run the algorithm on a real machine.

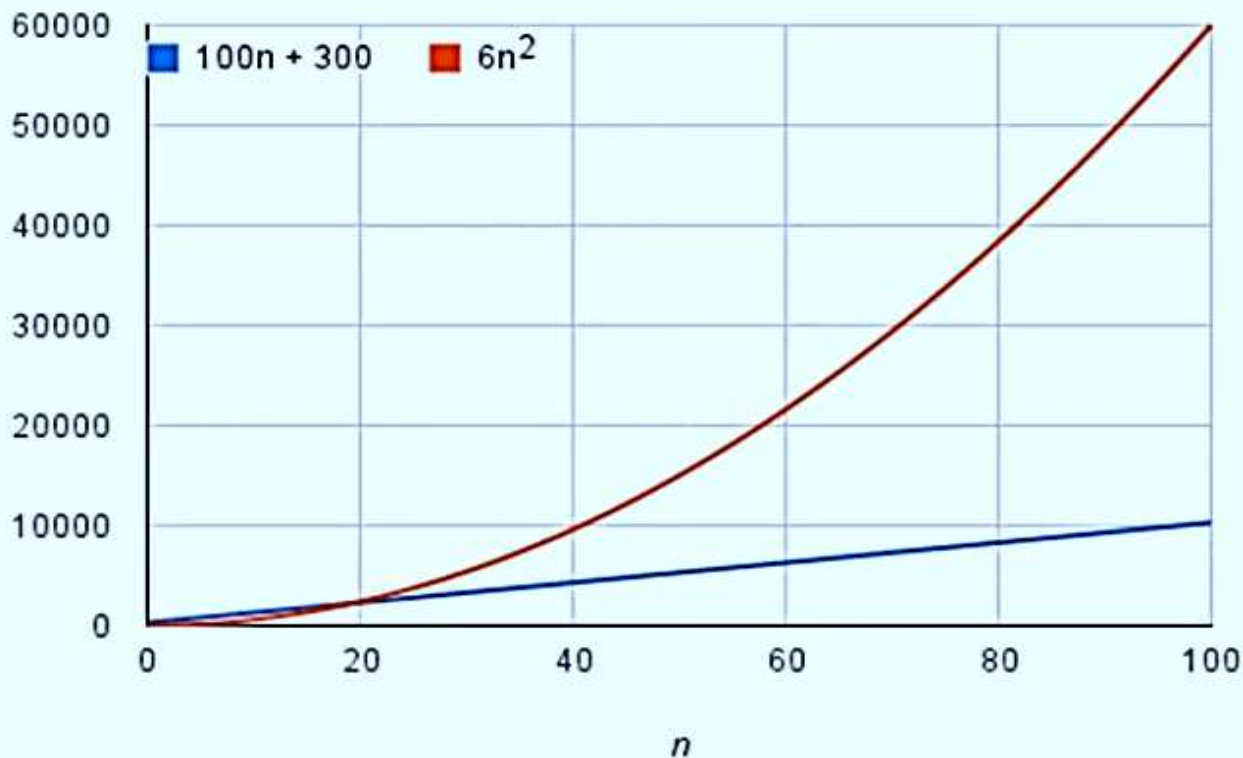


- Frequency count method

$$f(n): 6n^2 + 100n + 300$$

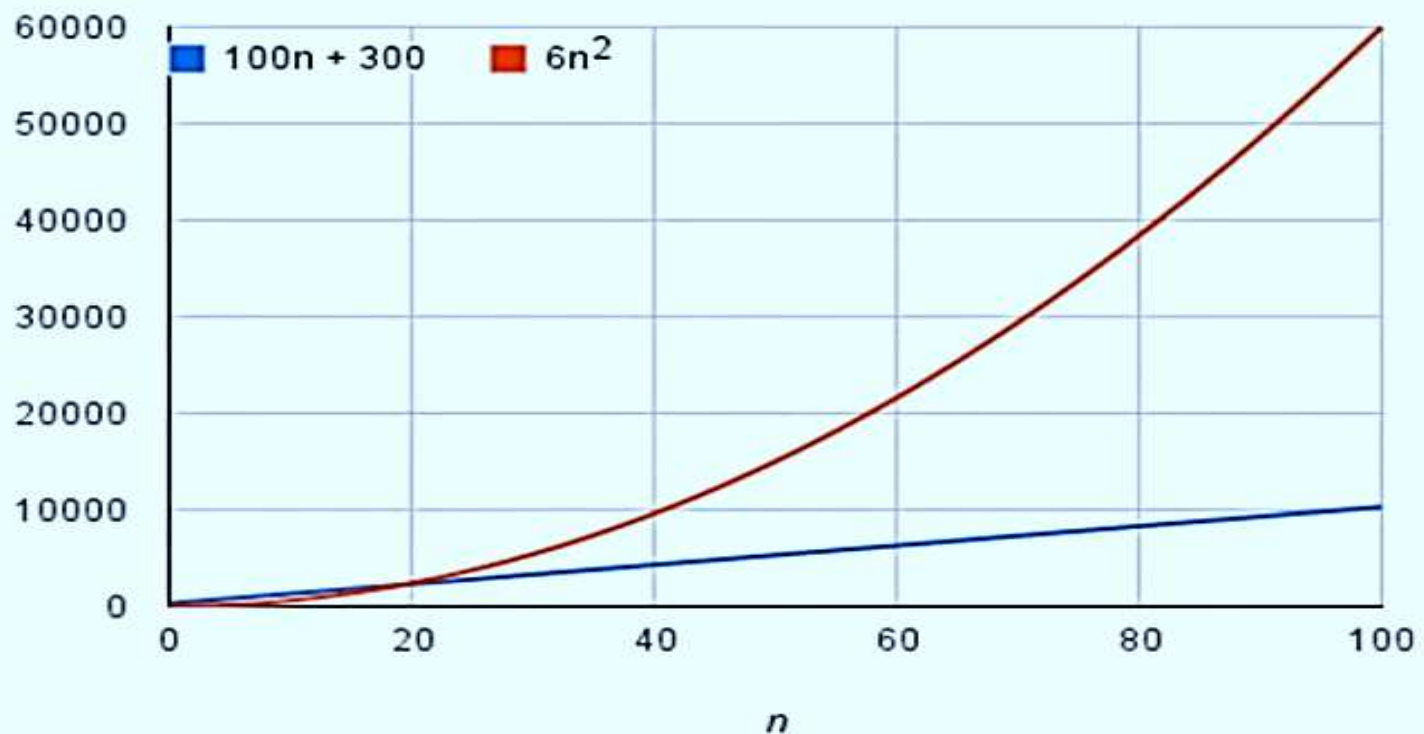
Asymptotic Analysis

$$f(n): 6n^2 + 100n + 300$$



Asymptotic Analysis

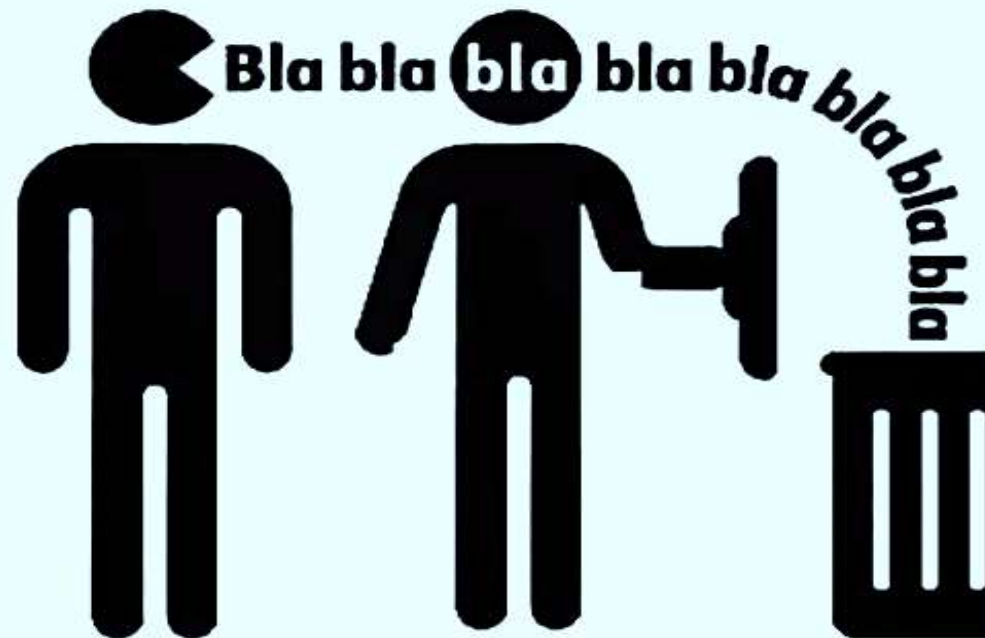
$f(n):$ n^2



Big **O** – Notation

$$f(n): 6n^2 + 100n + 300$$

$$f(n): 2n^2 + 150n + 330$$



Big **O** – Notation

$$\begin{array}{llll} f(n): 6n^2 + 100n + 300 & \longrightarrow & O(n^2) & \\ f(n): 2n^2 + 150n + 330 & \longrightarrow & O(n^2) & \end{array} \quad \left. \begin{array}{c} \text{---} \\ \text{---} \end{array} \right\} \longrightarrow O(n^2)$$
$$f(n): 2n^3 + 3n^2 + 100n \quad \longrightarrow \quad O(n^3)$$
$$f(n): 4\log_2(n) + 40 \quad \longrightarrow \quad O(\log_2(n))$$

- Used to measure the performance of any algorithm.
- By providing the order of growth of the function.

Big **O** – Notation

$$\begin{array}{llll} f(n): 6n^2 + 100n + 300 & \longrightarrow & O(n^2) & \\ f(n): 2n^2 + 150n + 330 & \longrightarrow & O(n^2) & \longrightarrow O(n^2) \\ f(n): 2n^3 + 3n^2 + 100n & \longrightarrow & O(n^3) & \\ f(n): 4\log_2(n) + 40 & \longrightarrow & O(\log_2(n)) & \end{array}$$

- Used to measure the performance of any algorithm.
- By providing the order of growth of the function.
- It Gives the upper bound on a function. **(Worst Case)**

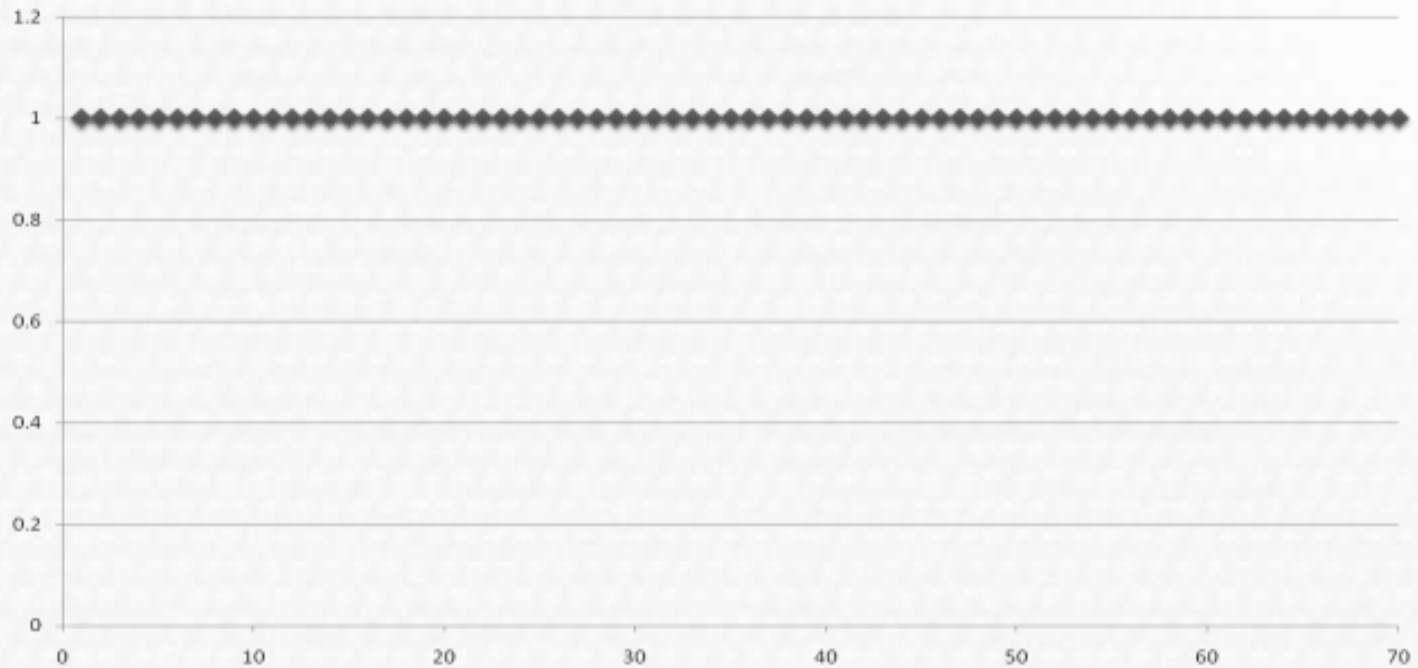
Rate of Growths

Time Complexity	Name	Example
1	Constant	Adding an element to the front of a linked list
$\log n$	Logarithmic	Finding an element in a sorted array
n	Linear	Finding an element in a unsorted array
$n \log n$	Linear Logarithmic	Sorting n items by 'Divide and Conquer'
n^2	Quadratic	Shortest path between 2 nodes in a graph
n^3	Cubic	Matrix Multiplication
2^n	Exponential	The Towers of Hanoi problem

Essential Seven functions to estimate algorithms performance

`Print("Hello Algorithms")`

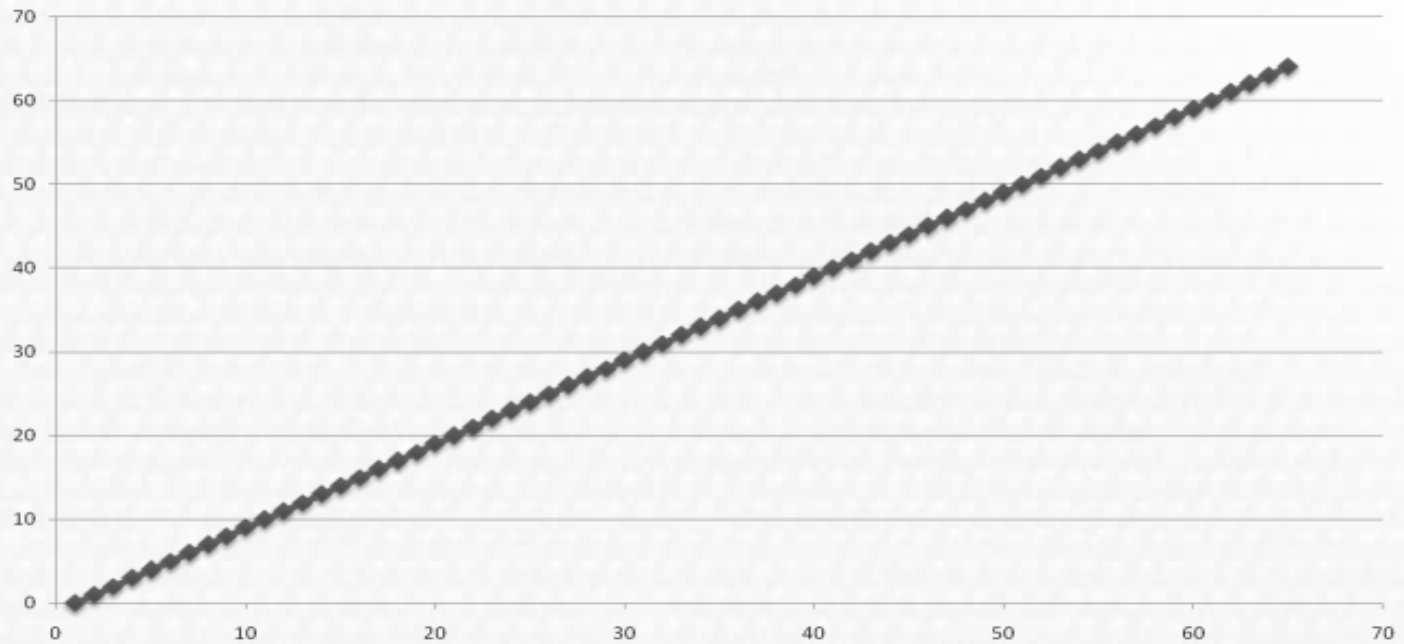
$$g(n) = 1$$



Essential Seven functions to estimate algorithms performance (cont'd)

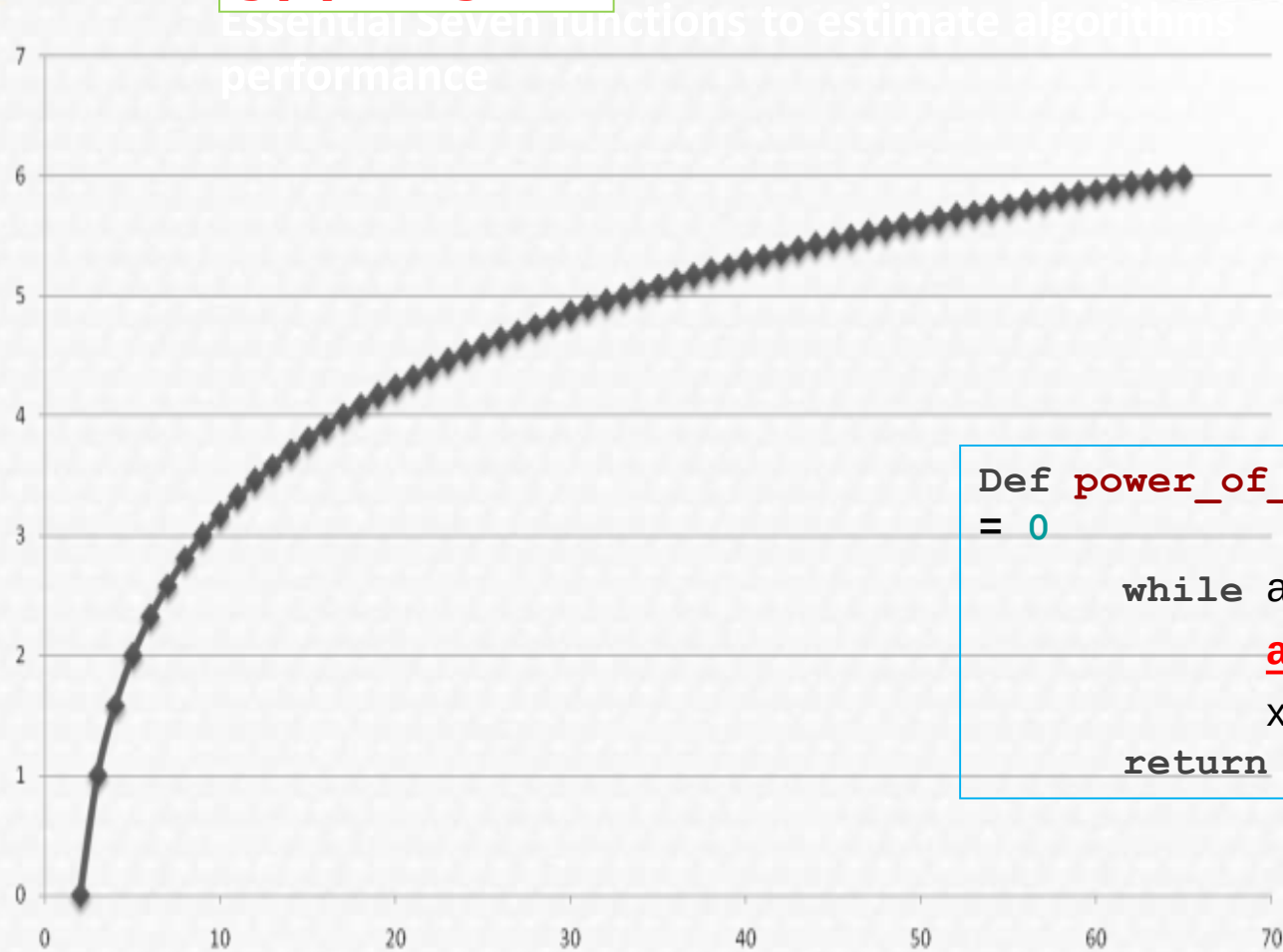
```
for i in range(0, n):  
    Print(i)
```

$$g(n) = n$$



Essential Seven functions to estimate algorithms performance (cont'd)

$$g(n) = \log n$$

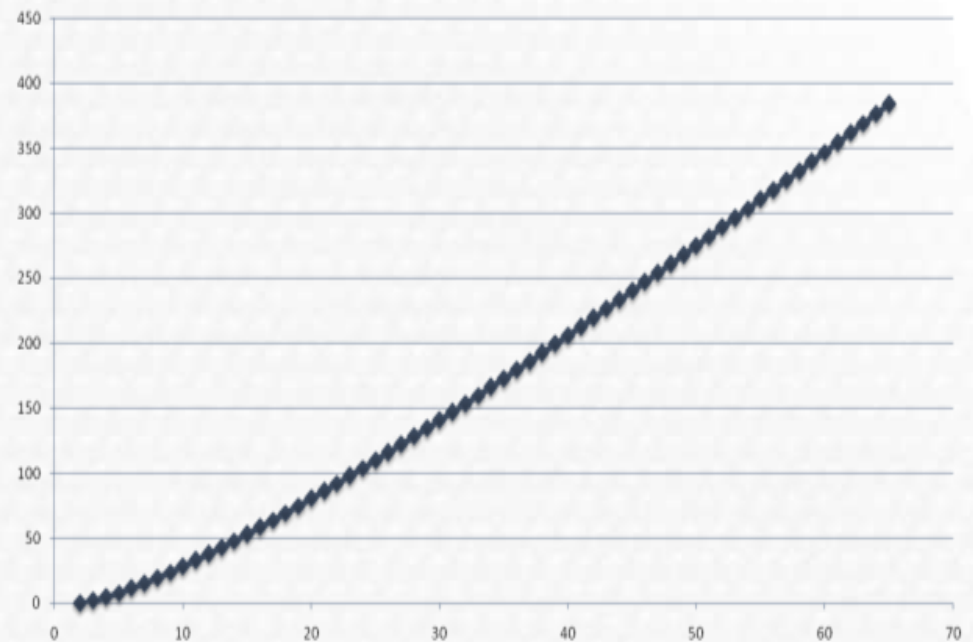


```
Def power_of_2(a) :      x
= 0
    while a > 1:
        a = a/2
        x = x+1
    return x
```

Essential Seven functions to estimate algorithms performance (cont'd)

```
for i in range(0,n):  
    Def power_of_2(a):  
        x = 0  
        while a > 1:  
            a = a/2  
            x = x+1  
        return x
```

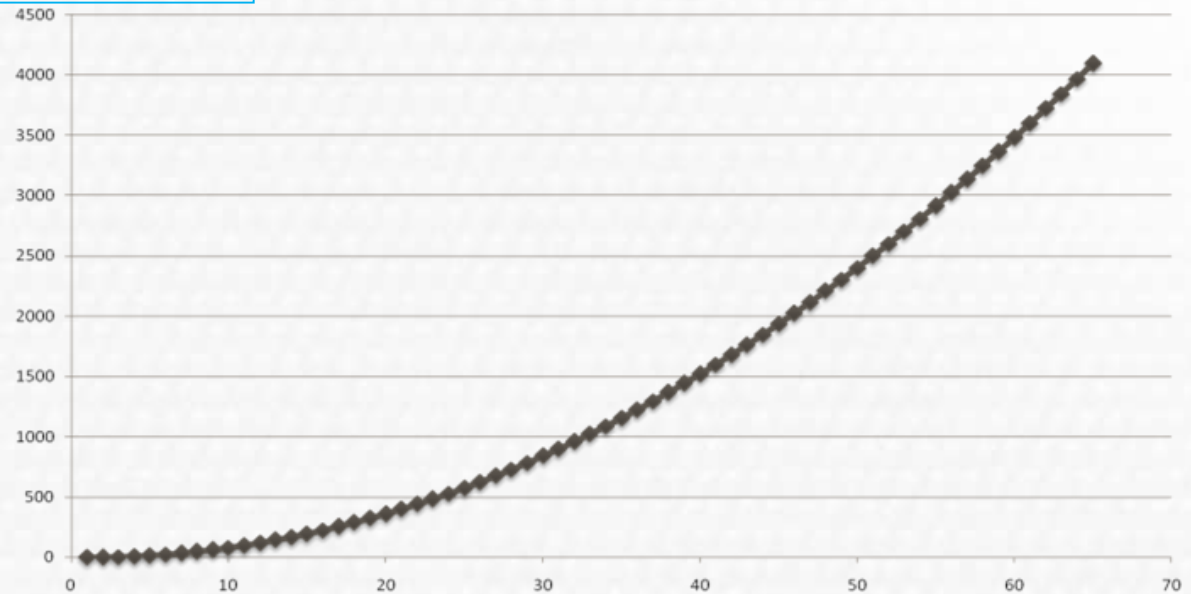
$$g(n) = n \lg n$$



Essential Seven functions to estimate algorithms performance (cont'd)

```
for i in range(0,n):  
    for j in range(0,n):  
        print(i*j);
```

$$g(n) = n^2$$

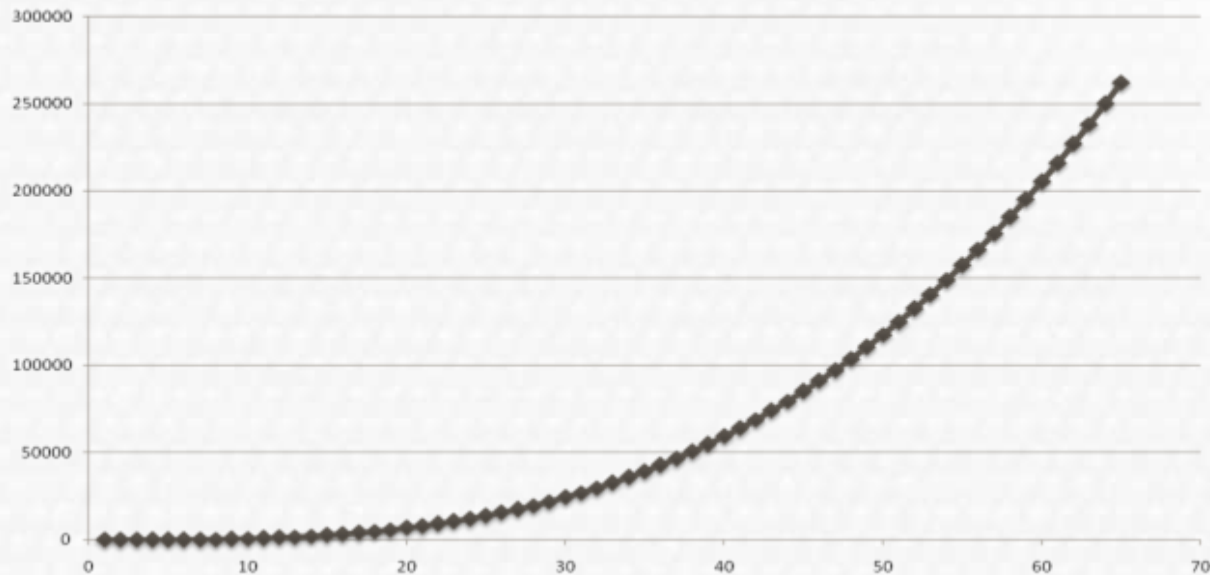


$$1 + 2 + 3 + \dots + (n-2) + (n-1) + n = \frac{n(n+1)}{2}.$$

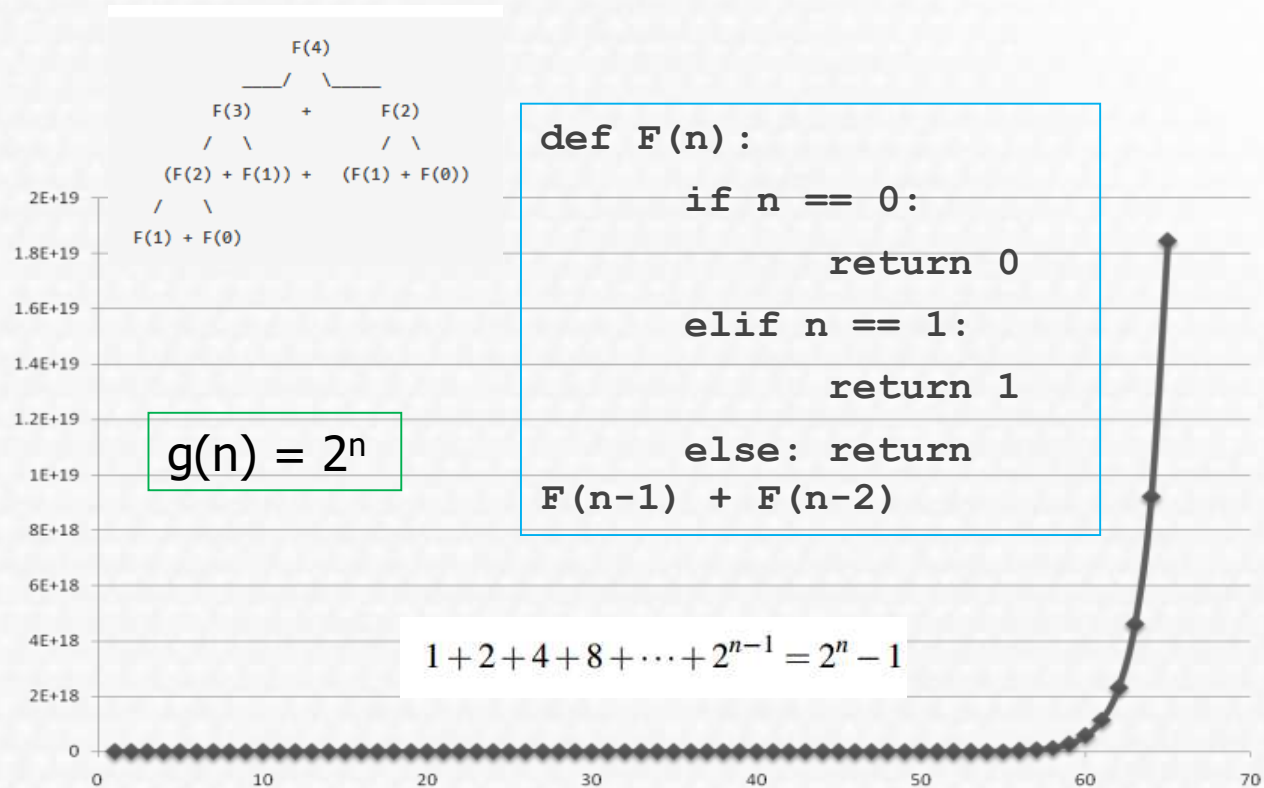
Essential Seven functions to estimate algorithms performance (cont'd)

```
for i in range(0, k):  
    for i in range(0,n):  
        for j in range(0,n):  
            print(i*j);
```

$$g(n) = n^3$$



Essential Seven functions to estimate algorithms performance (cont'd)



	<i>constant</i>	<i>logarithmic</i>	<i>linear</i>	<i>N-log-N</i>	<i>quadratic</i>	<i>cubic</i>	<i>exponential</i>
<i>n</i>	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4,096	65536
32	1	5	32	160	1,024	32,768	4,294,967,296
64	1	6	64	384	4,069	262,144	1.84×10^{19}

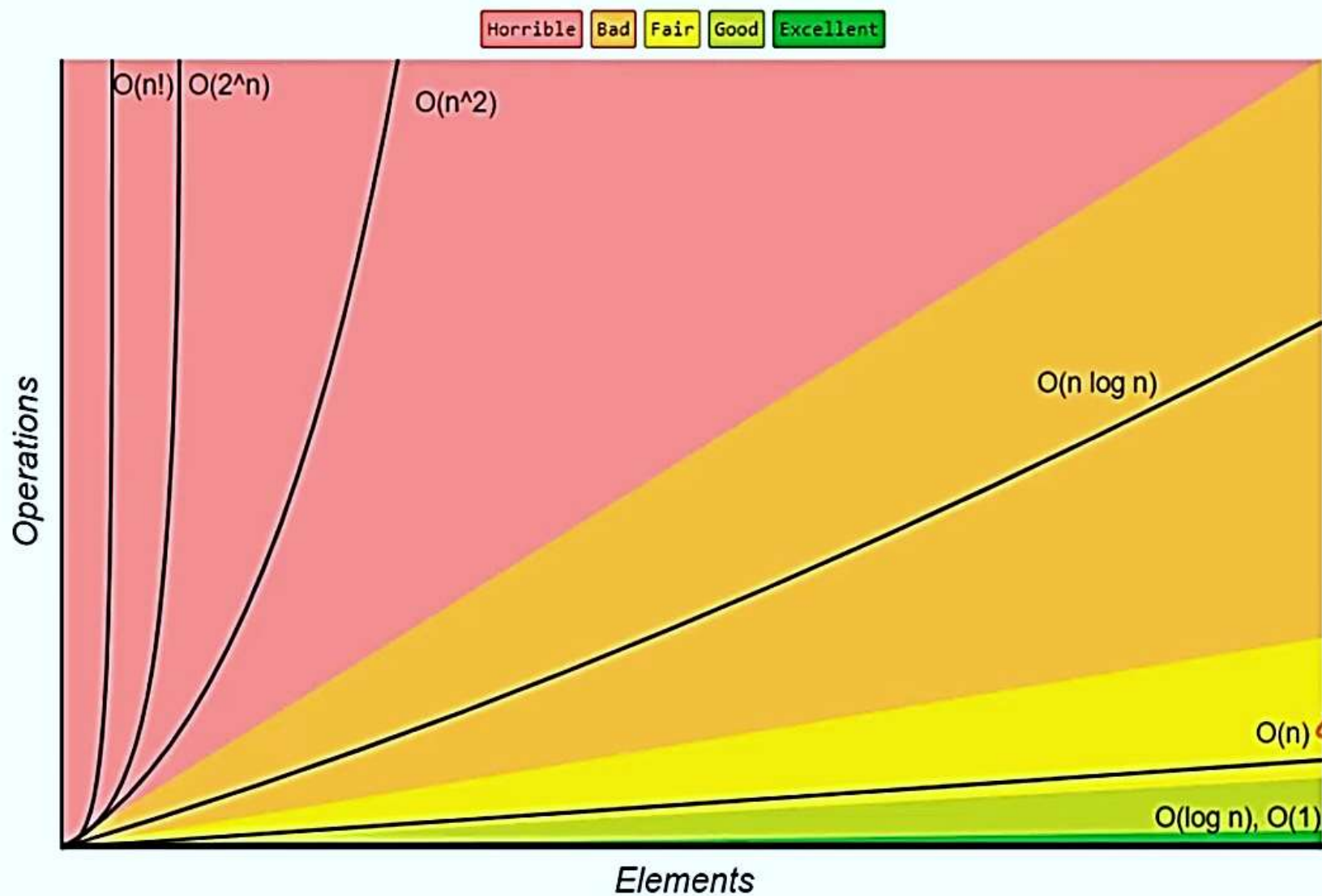
	<i>constant</i>	<i>logarithmic</i>	<i>linear</i>	<i>N-log-N</i>	<i>quadratic</i>	<i>cubic</i>	<i>exponential</i>
<i>n</i>	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4,096	65536
32	1	5	32	160	1,024	32,768	4,294,967,296
64	1	6	64	384	4,069	262,144	1.84×10^{19}

	<i>constant</i>	<i>logarithmic</i>	<i>linear</i>	<i>N-log-N</i>	<i>quadratic</i>	<i>cubic</i>	<i>exponential</i>
<i>n</i>	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4,096	65536
32	1	5	32	160	1,024	32,768	4,294,967,296
64	1	6	64	384	4,069	262,144	1.84×10^{19}

	<i>constant</i>	<i>logarithmic</i>	<i>linear</i>	<i>N-log-N</i>	<i>quadratic</i>	<i>cubic</i>	<i>exponential</i>
<i>n</i>	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4,096	65536
32	1	5	32	160	1,024	32,768	4,294,967,296
64	1	6	64	384	4,069	262,144	1.84×10^{19}

	<i>constant</i>	<i>logarithmic</i>	<i>linear</i>	<i>N-log-N</i>	<i>quadratic</i>	<i>cubic</i>	<i>exponential</i>
<i>n</i>	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4,096	65536
32	1	5	32	160	1,024	32,768	4,294,967,296
64	1	6	64	384	4,069	262,144	1.84×10^{19}

	<i>constant</i>	<i>logarithmic</i>	<i>linear</i>	<i>N-log-N</i>	<i>quadratic</i>	<i>cubic</i>	<i>exponential</i>
<i>n</i>	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4,096	65536
32	1	5	32	160	1,024	32,768	4,294,967,296
64	1	6	64	384	4,069	262,144	1.84×10^{19}



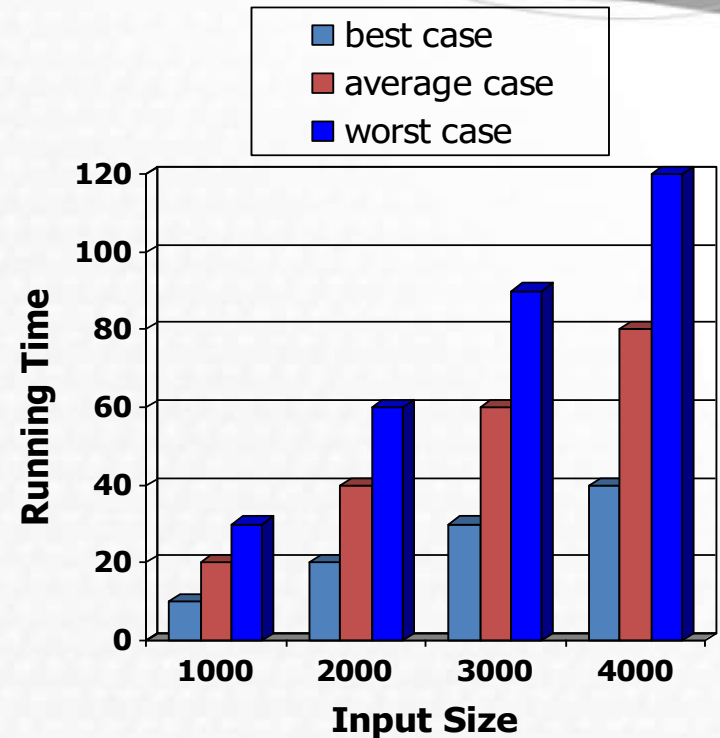
Asymptotic Notations

- Big O – Notation | (O) *(Worst Case)*
- Big Omega – Notation | (Ω) *(Best Case)*
- Big Theta – Notation | (Θ) *(Average Case)*



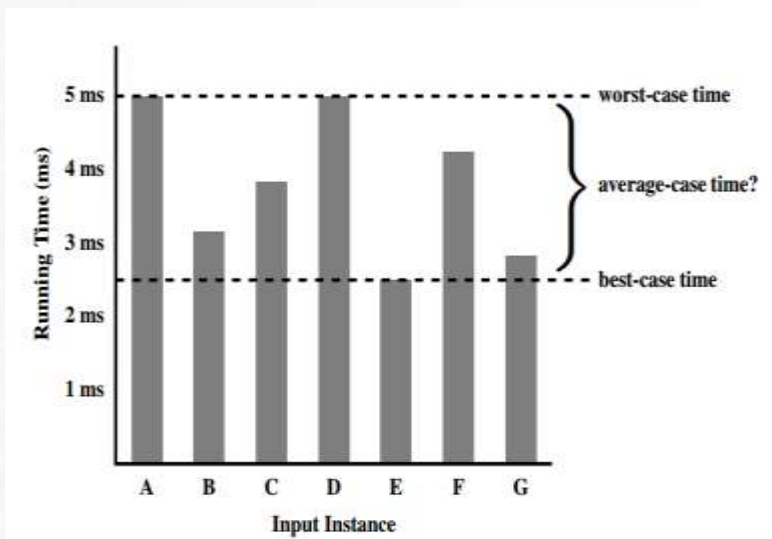
Running Time

- Most algorithms transform input objects into output objects.
- The running time of an algorithm typically grows with the **input size**.
- Average case time is often difficult to determine.
- We focus on the worst case running time.
 - Easier to analyze
 - Crucial to applications such as games, finance and robotics



Why discarding average case, And choose worst case instead?

- **An algorithm** may run faster on some inputs than it does on others of the same size. Thus, we may wish to express the running time of an algorithm as the function of the input size
- **Average-case** analysis is typically quite challenging. It requires us to define a probability distribution on the set of inputs, which is often a difficult task.
- **An average-case** analysis usually requires that we calculate expected running times based on a given input distribution, which usually involves sophisticated probability theory. Therefore we will characterize running times in terms of the **worst case**, as a function of the input size, n , of the algorithm.
- **Best-case** analysis is much easier than average-case analysis, as it requires only the ability to identify the worst-case input, which is often simple.



Let $f(n) = 2n^2 + 30n + 100$ Is $f(n) = O(n^2)$?

Let $f(n) = 2n^2 + 30n + 100$ Is $f(n) = O(n^2)$?

Definition:

$f(n) = O(g(n))$, there exist constants c, n_0 ,
such that $f(n) \leq c.g(n)$ for all $n \geq n_0$